Technische Hochschule Nürnberg

Deep Learning

Vanishing Gradient
Overfitting

Prof. Dr. Tobias Bocklet

Inhalte

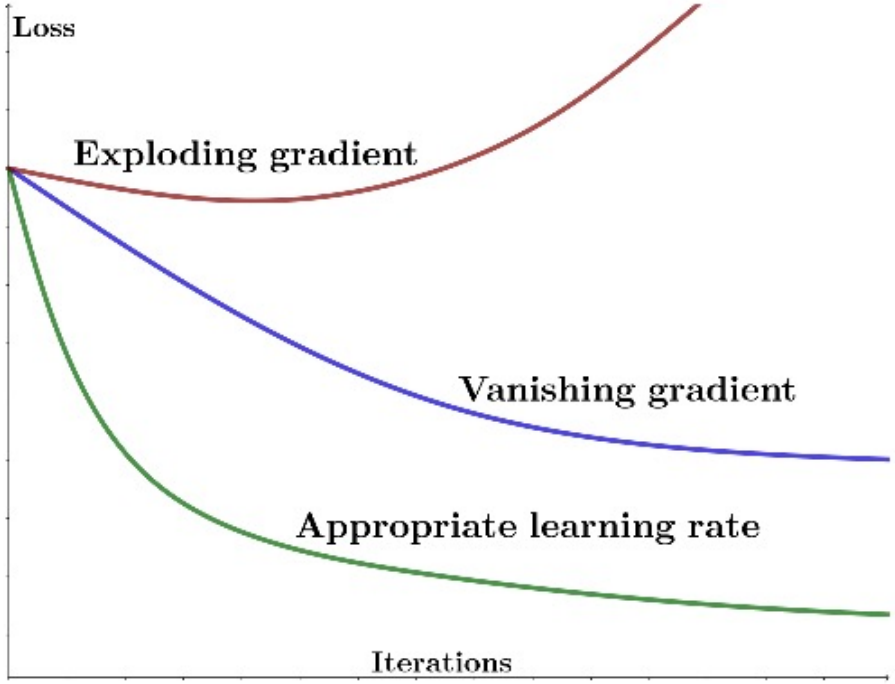
- Problem: Vanishing Gradient
  - Aktivierungsfunktionen
  - Initialisierungen

- Problem: Overfitting
  - Validierungsdaten
  - Regularisierung
  - Early-stopping
  - Dropout
  - Data Augmentation

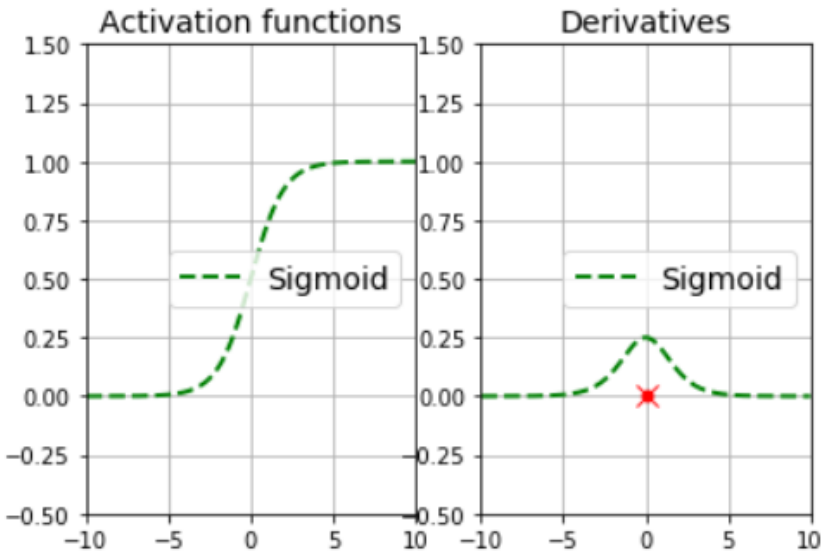
Wdh

- Warum ist eine Aktivierungsfunktion notwendig?
- Warum verwendet man in neuronalen Netzen keine Stufenfunktion zur Aktivierung?
- Was macht der Backpropagation-Algorithmus?
- Wie sieht ein neuronales Netz für Mehrklassen-Probleme aus?

Vanishing Gradient



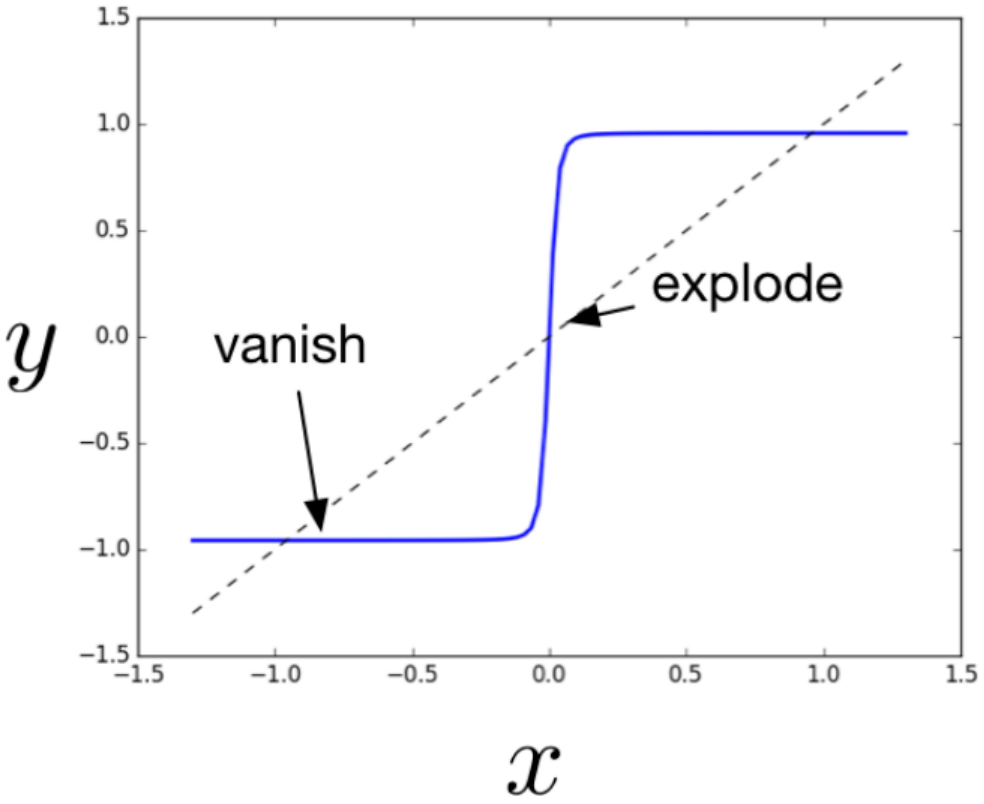
Aktivierungsfunktionen



- Sigmoid
  - Kennen wir schon
  - $g(x) = \frac{1}{1+e^{-x}}$
  - Nahe am biologischen Modell & differenzierbar
  - Wertebereich: $]0,1[$
  - Probabilistische Ausgabe
  - Saturierung an den Enden → Vanishing Gradient

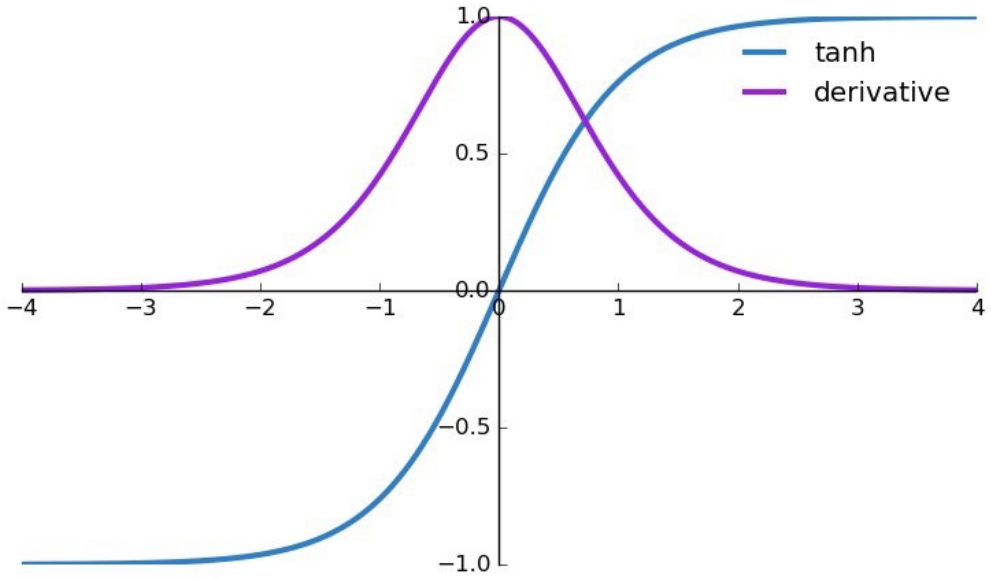
```
def sigmoid(x):
    return (1 / (1 + np.exp(-x)))
```

Vanishing Gradient



Aktivierungsfunktionen

- Tanh



  - Smooth
  - Wertebereich: $[-1,1]$
  - Saturierung an den Enden
  - Zentriert um 0

Aktivierungsfunktionen

- ReLU: Rectified Linear Unit
  - $ReLU(x) = \begin{cases} 0, & wenn\ x \leq 0 \\ x, & wenn\ x > 0 \end{cases}$

```
def relu(X):
    return np.maximum(0,X)
```

  - Einfach zu berechnen, weniger compute als sigmoid
  - Vermeidet „*Vanishing gradient*“-Problem
  - Kann *Dead Neurons* zur Folge haben
  - Wertebereich: $[0, \infty)$
  - Schnell (Speed up ~6x)
  - Keine Wskt: Verwenden in verdeckten Schichten

Aktivierungsfunktionen

- LeakyRELU: Exponential Linear Unit
  - $LeakyReLU(x) = \begin{cases} x & wenn\ x > 0 \\ \alpha x, & wenn\ x \leq 0 \end{cases}$

  - LeakyRELU "smoother" als ReLU
  - Vesuch *Dead Neurons* der ReLU zu minimieren
  - LeakyRELU kann negative Outputs erzeugen
  - Wertebereich: $(-\infty, \infty)$ → Gradienten können explodieren

```
def leakyrelu(z, alpha=0.1):
    return np.where(z > 0, z, z * alpha)
```

  - Parametric Leaky ReLU: α trainiert während Backprop

Aktivierungsfunktionen

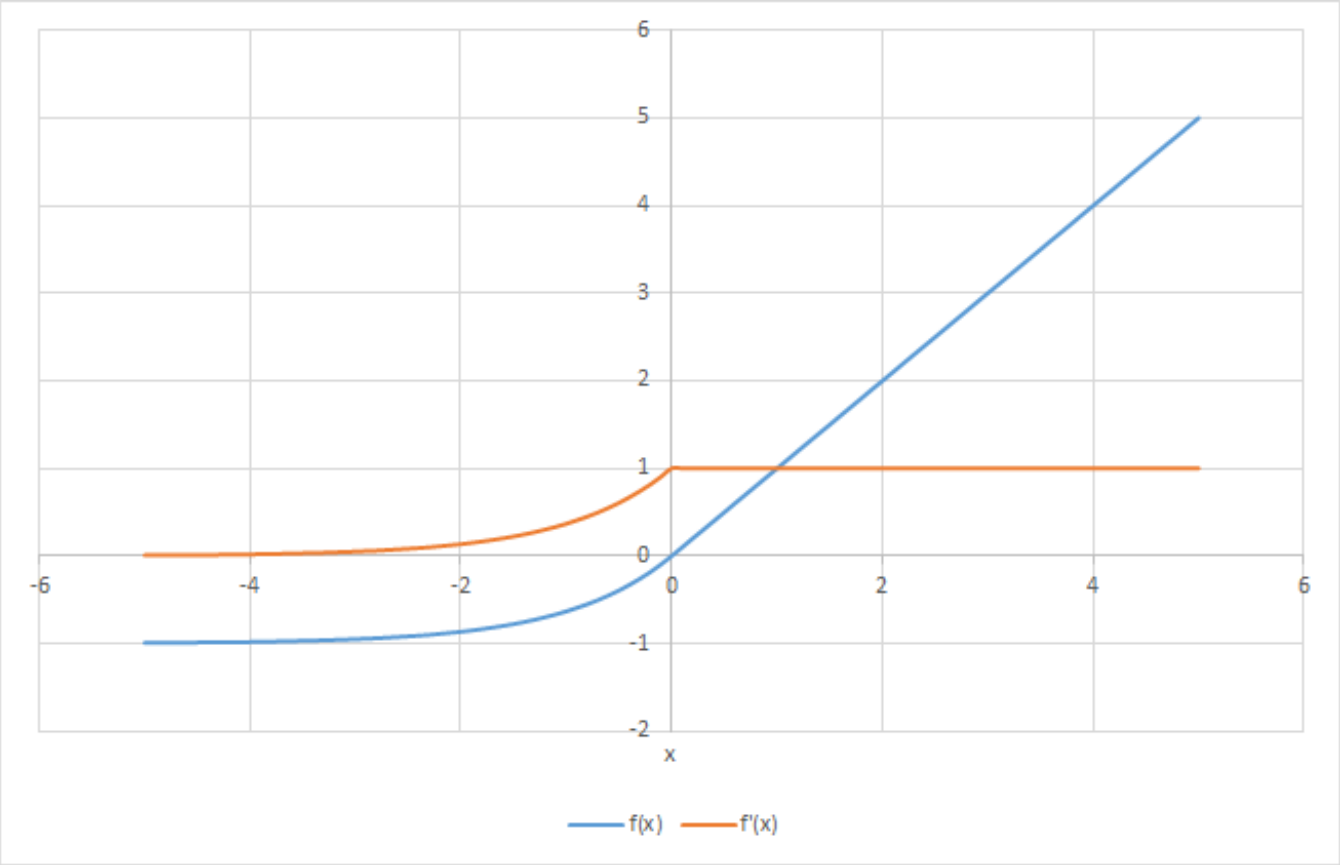
- ELU: Exponential Linear Unit
  - $ELU(x) = \begin{cases} x, & wenn\ x > 0 \\ \alpha * (e^x - 1), & wenn\ x \leq 0 \end{cases}$
  - ELU “smoother” als ReLU, nähert sich $-\alpha$ an
  - *Dead Neurons* werden vermieden
  - ELU kann negative Outputs erzeugen
  - Wertebereich: $(-\alpha, \infty)$, Mittelwert näher an 0 als ReLU

```
def elu(z,alpha):
    return z if z >= 0 else alpha*(e^z -1)
```

  - Nachteil: Langsamer als ReLU

Aktivierungsfunktionen

- ELU: Exponential Linear Unit



Aktivierungsfunktion/Kostenfunktion – Klassifikation

Zwei Klassen

- Einzelnes Neuron in Ausgabeklasse
- Aktivierungsfunktion in der Ausgabeschicht: sigmoid
- Aktivierungsfunktion in verdeckten Schichten: ReLU
- Kostenfunktion: binary cross entropy

Mehrere Klassen

- Ein Neuron pro Klasse in Ausgabeschicht(1-out-of-n coding, one-hot) in output layer
- activation output layer: Softmax
- activation hidden layers: ReLU
- loss function: cross entropy

Activation/Loss Function – Regression

- Anstatt diskreter Klassen: Vorhersage eines kontinuierlichen Wertes

- Einzelnes Neuron in Ausgabeschicht
- Aktivierung in der Ausgabeschicht: Linear
  (Summe der Gewichte wird weitergereicht)
- Aktivierung in den verdeckten Schichten: ReLU
- Kostenfunktion: Mean Squared Error

Gewichtsinitialisierung (1)

- Initialisierung mit 0: Kein Gradient, lernen nicht möglich
- Initialisierung aller Gewichte mit der selben Konstante:
  Neurone in den verdeckten Schichten fallen zu einem einzelnen Neuron zusammen
- Random Initialization: üblich, z.B.
  - Uniform im Interval $[-0.01;\ +0.01]$
  - Uniform im Interval $[-\frac{1}{\sqrt{n}};\ +\frac{1}{\sqrt{n}}]$,
    mit $n =$ # der Neurone der vorherigen Schicht

Gewichtsinitialisierung (2)

- Varianz der Eingabe eine Schicht gleich der Ausgabe (Analogie: Audio-Verarbeitungskette)
- Auch Gradienten müssen gleiche Varianz vor und nach eines Backprob-schrittes haben
- Mathematische Details: http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf
- Xavier/Glorot-Initialisierung:
  - Normalverteilt mit Mittelwert 0, $\sigma^2 = \frac{1}{(m+n)/2}$
  - Uniform im Interval $[-\frac{\sqrt{6}}{\sqrt{m+n}}; +\frac{\sqrt{6}}{\sqrt{m+n}}]$,
    mit m = #Neurone vorherige Schicht, n = #Neurone nachfolgende Schicht
  - Für sigmoid, bzw. tanh

Gewichtsinitialisierung (3)

- He-Initialisierung
  - Normalverteilt mit Mittelwert 0, $\sigma^2 = \frac{2}{n}$
  - Uniform im Interval $[-\frac{\sqrt{6}}{\sqrt{n}}; +\frac{\sqrt{6}}{\sqrt{n}}]$,
  - Für ReLU und Derivate
  - https://arxiv.org/pdf/1502.01852.pdf

Anmerkungen zu neuronalen Netzen

Vorteile:

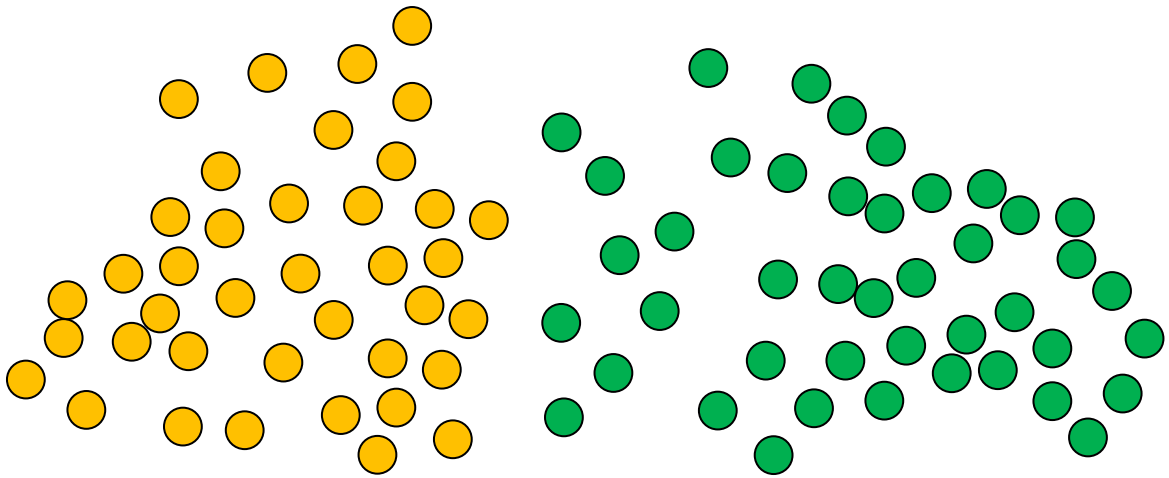
- Parallele Berechnungen möglich
- Kann durch Matrix-Multiplikationen formuliert werden
  → sehr gut geeignet für GPUs

Nachteile:

- # der Schichten und Neurone pro Schicht ist Design-Entscheidung
  - Theoretisches Minimum von 2/3 Schichten
    (mehr ist unter umständen besser, z.B. schneller Konvergenz)
  - Bestimmung der Parameter heuristisch → durch Experimente
- Hoher Rechenaufwand nötig
- Optimales Minimum durch nicht-lineare Optimierung nicht garantiert

Overfitting

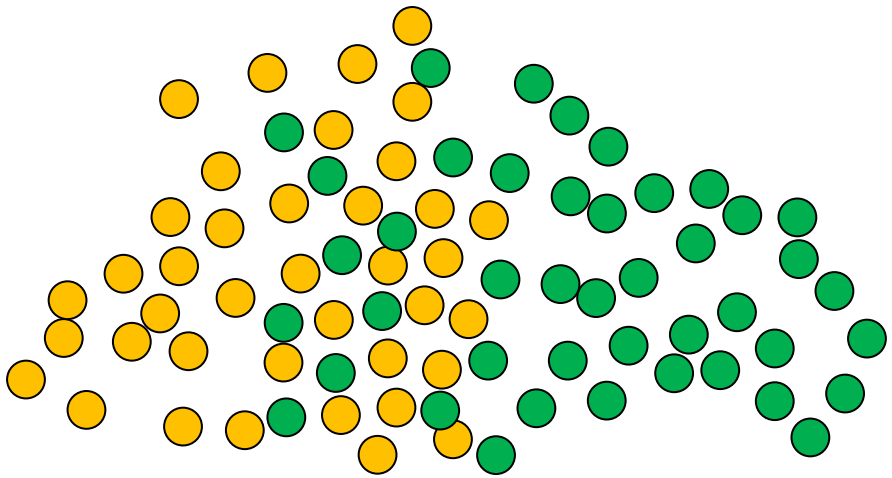
Klassifikation: 2 Klassenproblem



Trainingsbeispiel Klasse 1

Trainingsbeispiel Klasse 2

Klassifikation: 2 Klassenproblem



Trainingsbeispiel Klasse 1

Trainingsbeispiel Klasse 2

Overfitting in Words

> „Vier Parameter reichen mir, um einen Elefanten zu fitten, und mit fünf kann er noch mit dem Rüssel wackeln.“
>
> – JOHN VON NEUMANN

- Neuronale Netze haben (mindestens) tausende Parameter
- Super für uns: Komplexe Daten können modelliert werden
- Schlecht für uns: Overfitting

Overfitting

Overfitting: Zu viele Gewichte, die gelernte Hypothese wurde hervorragend an das Trainingset angepasst

$$C = \frac{1}{2M} \sum_{i=1}^{M} (h(x) - y)^2 \approx 0$$

Generalisiert aber nicht auf neuen, ungesehenen Daten

Wie beheben

- Niedrig dimensionaler Fall: Plotten der Hypothese mit Daten und Analyse

- Hochdimensionaler Fall:

  1. Anzahl der Merkmale reduzieren
     - Manuell, welche sind wichtig
     - Algorithmen zur Merkmalauswahl

  2. Regularisierung
     - Behalte alle Merkmale, aber reduziere Wertebereiche der Parameter ***w***
     - Bei vielen Merkmalen, die alle „ein wenig“ zur Vorhersage beitragen

  …

Zusätzliches Validierungsset

- Validierungsset bestimmen (z.B. split aus train-Daten)
- C während des Trainings auf Validierungsdatensatz berechnen
- Diese Entwicklung überwachen und mit Training vergleichen

Regularisierung

- Beispiel:

$$w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$

- w_3 und w_4 sollen klein werden
- Minimierung der Kostenfunktion: $\min_{\mathrm{w}} = \frac{1}{2M} \sum_{i=1}^{M} \left(h_w\left(x^{(i)}\right) - y^{(i)} \right)^2$

- Mit Regularisierung:

$$\min_{\mathrm{w}} = \frac{1}{2M} \sum_{i=1}^{M} \left(h_w\left(x^{(i)}\right) - y^{(i)} \right)^2 + 1000 w_3^2 + 1000 w_4^2$$

Regularisierung – Idee

- Kleine Werte für Parameter $w_0, \dots, w_N$
  - Einfachere Hypothesen
  - Weniger anfällig für Overfitting
- Hauspreise:
  - Merkmale: $x_0, \dots, x_{100}$
  - Gewichte: $w_0, \dots, w_{100}$

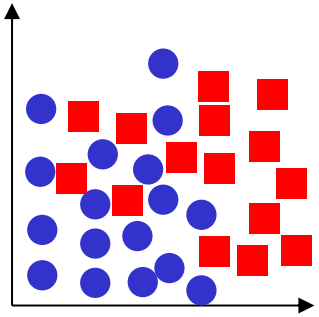
Tradeoff

$$\min_{\mathbf{w}} = \frac{1}{2M}\left(\sum_{i=1}^{M}\left(h_w\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda \sum_{j=1}^{N} w_j^2\right)$$

Passe ***w*** an Trainingsset an

Erzwinge kleine w_j

Regularisierung mit Kreuzentropie



- Kostenfunktion: $C = -\frac{1}{M}\sum_{i=1}^{M} y^i \log(h(x^{(i)})) + (1 - y^{(i)})\log(1 - h(x^{(i)}))$
- Mit Regularisierung:

$$C = -[\frac{1}{M}\sum_{i=1}^{M} y^i \log(h(x^{(i)})) + (1 - y^{(i)})\log(1 - h(x^{(i)}))\ ] + \lambda \sum_{j=1}^{N} w_j^2$$

$l_1\ vs.\ l_2$ Regularisierung neuronaler Netze

- l_2-Regularisierung

  → Regularisierung beschränkt Gewichte, hält diese klein

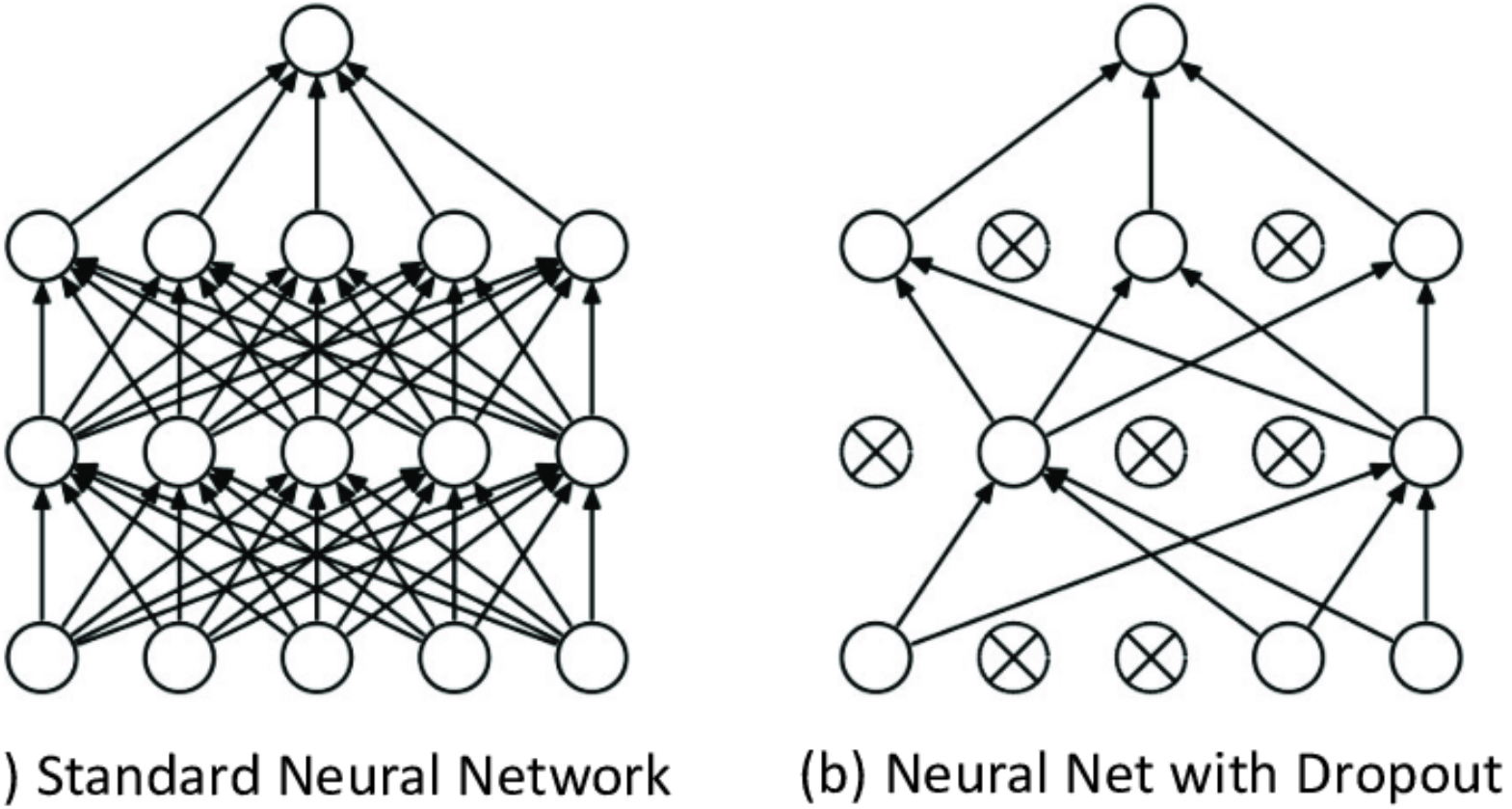
  ```
  keras.regularizers.l2(0.01)
  ```

- l_1-Regularisierung

  → Setzt Gewichte auf 0 erzeugt sparse models

  ```
  keras.regularizers.l1(0.01)
  ```

Dropout



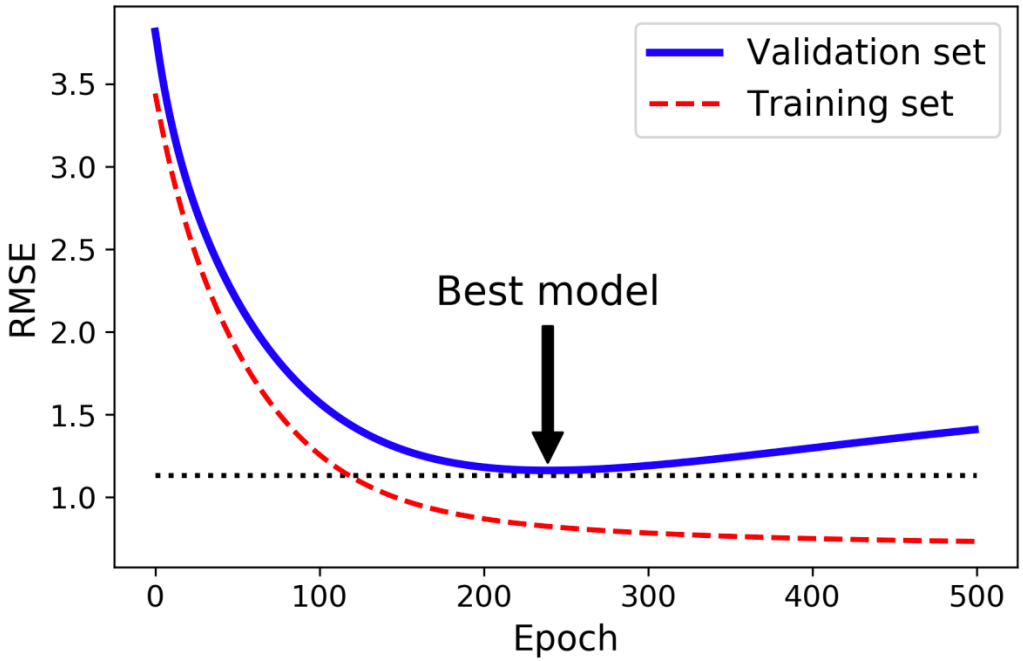
Dropout

- In jeder Iteration setze Output von Neuronen mit bestimmter Wahrscheinlichkeit randomisiert auf 0
- Typische Werte: 0.2-0.5 (=20-50% Entfernungswahrscheinlichkeit)
- Nur während des Trainings
- Neurone werden gezwungen weniger spezielle, sondern mehr abstrakte Konzepte zu Lernen

Early stopping

Early stopping:

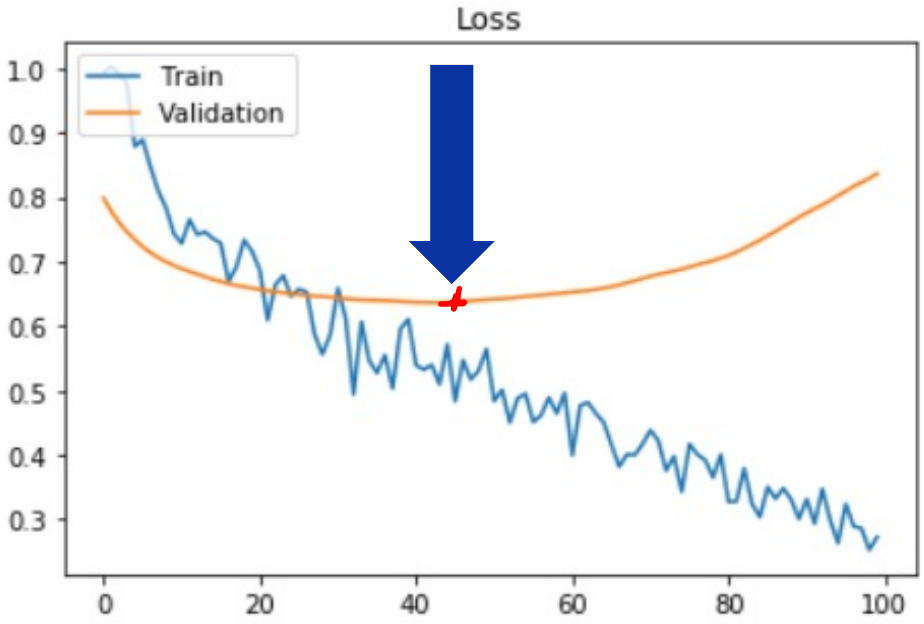
- Fehler C auf Validierungsdatensatz berechnen
- Training beenden wenn C auf diesen Daten steigt



Early stopping

Early stopping:

- Fehler C auf Validierungsdatensatz berechnen
- Training beenden wenn C auf diesen Daten steigt
- Implementiert in Keras mittels *Callback*



Early stopping

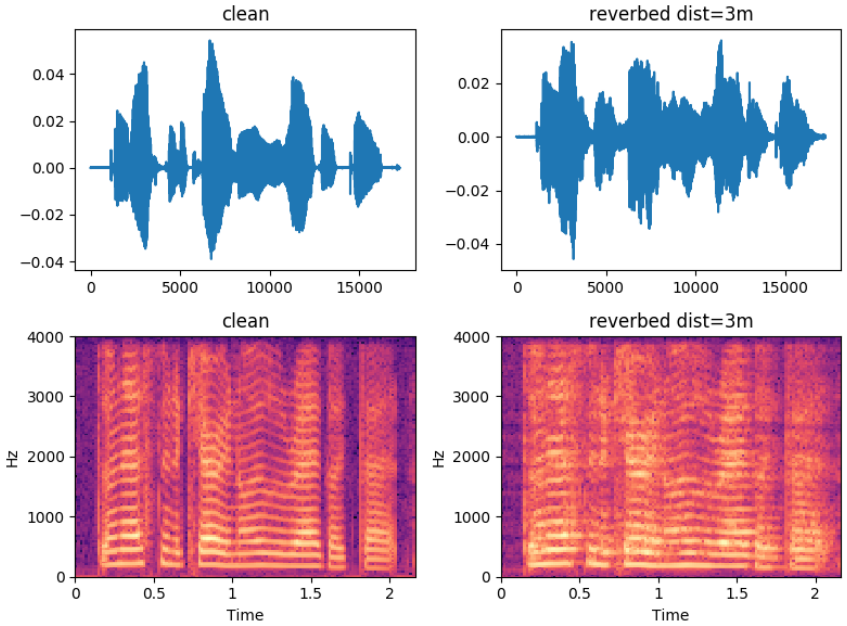
Early stopping:

- Fehler C auf Validierungsdatensatz berechnen
- Training beenden wenn C auf diesen Daten steigt
- Implementiert in Keras mittels *Callback*

min_delta	Minimum change in the monitored quantity to qualify as an improvement, i.e. an absolute change of less than min_delta, will count as no improvement.
patience	Number of epochs with no improvement after which training will be stopped.

Daten Augmentierung

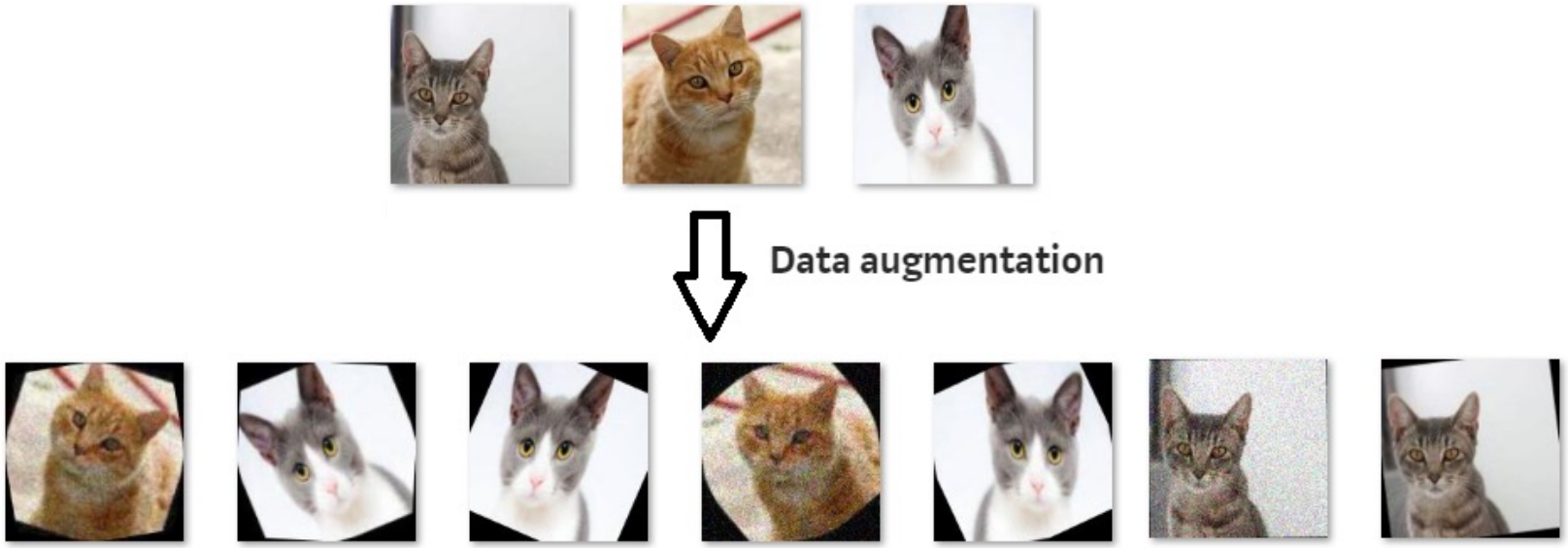
- Datenmenge künstlich vergrößern
- Aber wie?
- Variationen mit denen das System am Ende robust funktionieren soll



Audio/Sprache

- Reverb
- Hintergrundgräusche
- Time warping
- Evtl. Freq/time masking

Daten Augmentierung



Beispiel Bild

- Rauschen
- Drehung
- Flip (Horizontal oder Vertikal)

Zusammenfassung

- Problem: Overfitting
  - Validierungsdaten
  - Regularisierung
  - Early-stopping
  - Dropout
  - Data Augmentation
- Problem: Vanishing Gradient
  - Aktivierungsfunktionen
  - Initialisierungen

Praktische Hilfestellung (default config):

- Initializer: He-init
- Aktivierung in Hidden Schichten: (R)eLU
- Ausgabeschicht: Sigmoid oder Softmax bei Klassifikation
- Normalisierung: Keine wenn flach, BatchNorm wenn tief

Next on Maschine Learning…

- Optimiertes Training
  - Momentum
  - RMSprop
  - Adagrad
  - Adam

- Convolutional Neural Networks